Data Governance Copilot

Interview Question Bank

Topic 12 — Human-in-the-Loop (HITL)

12 questions · 4 sections · Two-Pass Pattern · approved flag · HITL Keywords · Streamlit · Teams Adaptive Cards · Race Conditions

ndational Core concepts | Intermediate Design decisions | Advanced Deep internals | Gotcha Real bugs / traps

---

The Two-Pass auto_ticket_node Pattern

Q01 Foundational

What is the HITL pattern in your project and why is human approval needed before creating Jira tickets? Why not just auto-create them?

Hint: Think about the governance context and the risk of false positives.

Key points to cover:

- HITL (Human-in-the-Loop): a human must explicitly approve an action before the system executes it — a safety gate on autonomous write operations
- Why not auto-create: data anomaly detection is probabilistic — a 'retention dropped 15%' signal could be a real incident or a data pipeline delay
- Auto-creating a Jira incident for a false positive: wastes ops team time, creates noise, erodes trust in the system
- Governance context: Jira tickets are formal incident records — they trigger SLA clocks, escalation policies, and audit trails; must not be created frivolously
- HITL keywords signal high-stakes situations: 'threshold', 'missing', 'below', 'risk', 'drop', 'fail' — these indicate anomalies worth human review
- Human approval adds accountability: the approver is on record — 'John approved incident ticket creation at 14:32 UTC' is an audit trail entry
- Contrast with read operations: information_node, knowledge_node, metadata_node run fully autonomously — reading data carries no write risk
- Design principle: autonomous execution is fine for reads; all writes (Jira, Collibra, rule registry) should be HITL-gated in a governance system

---

Q02 Foundational

Walk through the two-pass execution of auto_ticket_node step by step. What does each pass do and what state fields are involved?

Hint: Trace the exact state transitions in both the first and second invocation.

Key points to cover:

- PASS 1 — Detection (approved=False, default): auto_ticket_node checks state['anomalies'] for HITL keywords (threshold, missing, below, risk, drop, fail)
- If keywords found: build pending_action dict: {'action': 'create_jira_ticket', 'anomalies': [...], 'suggested_priority': 'High'} → write to state['pending_action']
- Pass 1 does NOT create any Jira ticket — it only sets the pending_action and returns; graph continues to synthesizer_node
- synthesizer_node includes the pending_action in final_summary: 'Anomaly detected. Pending action requires approval: create Jira incident for retention drop'
- User sees the response, reviews the anomaly, and decides to approve or reject via Streamlit panel / Teams card / MCP approve_action tool
- PASS 2 — Execution (approved=True): new graph invocation on same thread_id with approved=True in state
- auto_ticket_node re-runs: sees approved=True AND pending_action is set → calls CapacityAgent (Jira REST) to create the ticket
- After ticket creation: clear state['pending_action'] = None, append ticket ID to agent_results, return — no more pending actions

```
# auto_ticket_node two-pass logic def auto_ticket_node(state: AgentState) -> AgentState: anomalies = state.get('anomalies', []) approved = state.get('approved', False) pending = state.get('pending_action') # Pass 1: detect anomalies, set pending if not approved and _has_hitl_keywords(anomalies): return {**state, 'pending_action': { 'action': 'create_jira_ticket', 'anomalies': anomalies }} # Pass 2: execute if approved if approved and pending: ticket = capacity_agent.create_ticket(pending) return {**state, 'auto_tickets': [ticket], 'pending_action': None} return state # no anomalies — pass through
```

Q03 Intermediate

What are the HITL keywords and how does the detection logic work? Could a false negative (missed anomaly) occur?

Hint: Understand both the keyword list and its limitations.

Key points to cover:

- HITL keywords: 'threshold', 'missing', 'below', 'risk', 'drop', 'fail' — checked against state['anomalies'] list (strings from agent execution)
- _has_hitl_keywords(anomalies): joins anomaly strings, lowercases, checks if any keyword is a substring — case-insensitive match
- False negative scenario 1: anomaly described as 'retention declined significantly' — 'declined' is not in the keyword list; HITL not triggered; ticket auto-skipped
- False negative scenario 2: agent returns anomaly in a different language or abbreviation — 'ret. metric below SLA' might match 'below' but depends on how anomaly is formatted
- False negative scenario 3: anomaly is in agent_results.data dict but not surfaced to state['anomalies'] — node must explicitly append to anomalies for HITL to see it
- Fix for false negatives: LLM-based anomaly classification — instead of keyword matching, ask the LLM 'does this anomaly require human review? yes/no'
- Fix for missing anomalies: each agent node should explicitly populate state['anomalies'] via Annotated[List, operator.add] — same pattern as agent_results and sources
- Monitoring: track HITL trigger rate — if it drops to 0% for a week, either no anomalies occurred OR the keyword detection is broken

---

Q04 Advanced

What happens if a user never approves or rejects a pending HITL action? How long does the pending_action sit in the checkpoint and what operational problems does this cause?

Hint: This is about stale HITL state management — a real production gap.

Key points to cover:

- Current behaviour: pending_action sits in the PostgresSaver checkpoint indefinitely — no TTL, no expiry, no cleanup
- Problem 1: the next query on the same thread_id re-enters auto_ticket_node with pending_action still set — if approved=False (default), Pass 1 re-evaluates and may overwrite pending_action with a new anomaly
- Problem 2: user returns days later, approves a stale pending_action — Jira ticket created for an anomaly that has already been resolved or superseded
- Problem 3: storage accumulation — millions of stale pending_actions in checkpoints over months
- Fix 1: pending_action TTL — store created_at timestamp in pending_action dict; auto_ticket_node rejects approval if pending_action is older than 24 hours
- Fix 2: explicit rejection path — if user submits a new query without approving, auto-reject the old pending_action and log the rejection
- Fix 3: nightly_refresh_dag task — query checkpoints for threads with pending_action older than 48h → auto-reject + notify the original requestor
- Fix 4: MCP get_system_status tool — expose 'pending_hitl_count' metric; ops alert if > threshold (indicates stale approvals accumulating)

Streamlit HITL Panel

Q05 Intermediate

Explain how the Streamlit HITL panel works. How does it detect a pending action, render the panel, and submit the approval?

Hint: Trace the Streamlit execution flow from query response to approval submission.

Key points to cover:

- _render_hitl_panel() is called after every graph invocation in the Streamlit UI — checks st.session_state.pending_hitl
- After _run_graph() returns: if result['pending_action'] is not None → set st.session_state.pending_hitl = result['pending_action']
- Panel renders: displays anomaly details from pending_hitl dict (anomaly description, suggested priority, affected data products)
- Two buttons: 'Approve' and 'Reject' — rendered with st.button() — clicking triggers a Streamlit re-run
- Approve button handler: calls graph.invoke() with {'approved': True, 'thread_id': current_thread_id} — Pass 2 executes
- Reject button handler: clears st.session_state.pending_hitl, calls graph with {'approved': False, 'pending_action': None} to clean up state
- After approval: _render_hitl_panel() finds st.session_state.pending_hitl is None — panel disappears from UI
- Streamlit re-run model: every button click triggers full script re-run; session_state persists pending_hitl across re-runs — critical for HITL panel continuity

---

Q06 Intermediate

What is st.session_state and why is it critical for the HITL panel? What would break if you used a regular Python variable instead?

Hint: Streamlit's execution model makes this non-obvious — explain it clearly.

Key points to cover:

- Streamlit reruns the entire Python script from top to bottom on every user interaction (button click, slider move, text input)
- Regular Python variables: re-initialised to their default on every rerun — pending_hitl = None at the top of the script means the HITL state is lost on every click
- st.session_state: persists across reruns within the same browser session — values survive the script re-execution
- Without session_state: user sees the HITL panel, clicks Approve — Streamlit reruns — pending_hitl is None again — panel disappears before the approval graph call fires
- With session_state: Approve click sets st.session_state.approved = True, triggers rerun — script reads session_state.approved, fires Pass 2, clears pending_hitl
- Thread safety: each browser tab has its own session_state — two users on separate tabs have independent HITL states (correct behaviour)
- Limitation: session_state is in-memory per uvicorn worker process — if Streamlit restarts, session_state is lost; HITL pending state must be recovered from PostgresSaver checkpoint
- Recovery pattern: on app startup, if a thread_id cookie exists, reload the checkpoint and check for pending_action — restore st.session_state.pending_hitl if found

---

Teams Adaptive Cards & HMAC Flow

Q07 Intermediate

How does the Teams bot deliver and process HITL approval? Walk through the full flow from anomaly detection to Approve button click in Teams.

Hint: The Teams HITL flow is distinct from Streamlit — explain the webhook round-trip.

Key points to cover:

- 1. User sends query in Teams channel → Teams webhook → bot.py routes to _run_graph() equivalent
- 2. Graph runs, auto_ticket_node detects anomaly, sets pending_action — Pass 1 complete
- 3. bot.py receives the graph result, detects pending_action is set → calls build_hitl_card(pending_action)
- 4. build_hitl_card() constructs a Teams Adaptive Card JSON with: anomaly description, suggested priority, two Action.Submit buttons ('Approve' and 'Reject')
- 5. Bot sends the Adaptive Card as a reply message in the Teams channel — all channel members can see it
- 6. User clicks 'Approve' button → Teams sends a new webhook POST to /teams/webhook with activity type 'invoke' and action data {'action': 'approve', 'thread_id': '...'}
- 7. bot.py receives the invoke activity, extracts action data, calls graph.invoke() with approved=True on the thread_id — Pass 2 executes
- 8. Jira ticket created → bot sends a confirmation message/card with the ticket URL
- HMAC verification runs on both the initial webhook AND the Approve invoke — every Teams request is authenticated

---

Q08 Intermediate

Explain the structure of the Adaptive Card for HITL. What fields does build_hitl_card() include and how does Action.Submit carry data back to your webhook?

Hint: Teams Adaptive Cards have a specific JSON schema — show you understand it.

Key points to cover:

- Adaptive Card is a JSON payload sent as a Teams message attachment — renders natively in Teams desktop and mobile
- Card body includes: TextBlock for anomaly title ('Governance Alert: Retention Anomaly Detected'), FactSet for details (anomaly description, affected products, detected at timestamp, suggested Jira priority)
- Action buttons: Action.Submit type — when clicked, Teams posts the card's data property back to your webhook
- Approve button: Action.Submit with data={'action': 'approve', 'thread_id': thread_id, 'pending_action_id': id}
- Reject button: Action.Submit with data={'action': 'reject', 'thread_id': thread_id}
- thread_id in the card data is critical: the webhook must know which LangGraph thread to resume on approval
- bot.py on invoke activity: reads activity.value (the submitted data dict), extracts action and thread_id, routes to approve or reject handler
- Card update after action: bot calls Teams API to update the original card — replaces Approve/Reject buttons with 'Approved by John at 14:32' status to prevent double-clicking

```
# cards.py - Adaptive Card structure def build_hitl_card(pending_action: dict, thread_id: str) -> dict: return { 'type': 'AdaptiveCard', 'version': '1.4', 'body': [ {'type': 'TextBlock', 'text': 'Governance Alert', 'weight': 'Bolder', 'size': 'Medium'}, {'type': 'FactSet', 'facts': [ {'title': 'Anomaly', 'value': pending_action['anomalies'][0]}, {'title': 'Priority', 'value': pending_action.get('suggested_priority','High')}, ]} ], 'actions': [ {'type': 'Action.Submit', 'title': 'Approve', 'data': {'action': 'approve', 'thread_id': thread_id}}, {'type': 'Action.Submit', 'title': 'Reject', 'data': {'action': 'reject', 'thread_id': thread_id}}, ] }
```

Q09 Advanced

What happens if two Teams users both click Approve on the same HITL card simultaneously? Walk through the race condition and how you fix it.

Hint: Concurrent approvals are a real multi-user Teams scenario.

Key points to cover:

- Race condition: User A and User B both see the Adaptive Card and both click Approve within milliseconds of each other
- Two webhook POSTs arrive at /teams/webhook simultaneously — both pass HMAC verification
- Both handlers call graph.invoke(approved=True, thread_id=X) concurrently — two parallel LangGraph invocations on the same thread
- Both invocations see pending_action set, both call CapacityAgent.create_ticket() → two identical Jira tickets created
- Fix 1: Redis distributed lock — before Pass 2, acquire a Redis lock keyed on thread_id with 30s TTL; second request sees lock and returns 'already approved'
- Fix 2: optimistic locking — store a pending_action_version in the checkpoint; Pass 2 only proceeds if version matches the card's version; second approval fails version check
- Fix 3: card update as idempotency gate — immediately update the Teams card to show 'Processing...' when first Approve is clicked; second click on a disabled card sends no webhook
- Fix 4: Jira idempotency key — include thread_id + pending_action_id as Jira custom field; second create call finds existing ticket and returns it rather than creating duplicate
- Best answer: combine Fix 1 (Redis lock, fast) + Fix 4 (Jira idempotency, durable fallback)

Design Depth & Production Considerations

Q10 Advanced

Your HITL pattern requires the user to make a second request (or button click) to execute the approved action. How does LangGraph know to resume from auto_ticket_node rather than re-running the full graph from pre_hook?

Hint: This is a deep question about LangGraph checkpoint resume semantics.

Key points to cover:

- LangGraph checkpointing saves state after EVERY node execution — the checkpoint after Pass 1 contains the full AgentState including pending_action and all agent_results
- Pass 2 invocation: graph.ainvoke({'approved': True}, config={'configurable': {'thread_id': same_id}}) — LangGraph loads the latest checkpoint for that thread_id
- The loaded state already has: intent, next_agents, agent_results, sources from Pass 1 — these are not re-computed
- LangGraph doesn't re-run from START automatically — it merges the new input ({'approved': True}) into the loaded checkpoint state
- However: the graph does re-run from START with the merged state — pre_hook, supervisor_node, all agent_nodes execute again
- Why this is OK: @cached_node returns immediately for information/knowledge/metadata nodes — cache hits mean no real re-computation
- auto_ticket_node: now sees approved=True AND pending_action from checkpoint → executes Pass 2 — creates the Jira ticket
- Improvement: LangGraph interrupt_before / interrupt_after API (newer feature) — pauses graph AT auto_ticket_node, resumes exactly there; avoids re-running upstream nodes even with cache

---

Q11 Gotcha

A developer adds a new agent that runs after auto_ticket_node. The HITL approval now also re-runs this new agent. Is this a bug or expected behaviour and how do you fix it if it's unwanted?

Hint: Graph re-run on approval has side effects on downstream nodes.

Key points to cover:

- Expected or bug: depends on the new agent's behaviour — if it's a read-only reporting agent, re-running is harmless (cached); if it's another write agent, re-running is dangerous
- Scenario: new_audit_agent runs after auto_ticket_node and writes an audit log entry — Pass 2 approval causes BOTH a Jira ticket AND a duplicate audit log entry
- Root cause: Pass 2 re-runs the full graph including nodes after auto_ticket_node — these weren't gated on the approval
- Fix 1: check _agents_ran in new_audit_agent — if already in _agents_ran (from Pass 1 checkpoint), skip execution; only run if genuinely a new invocation
- Fix 2: separate HITL graph from main graph — main graph runs to auto_ticket_node and pauses; HITL resume graph starts FROM auto_ticket_node; downstream agents are graph-2 exclusive
- Fix 3: use LangGraph interrupt_before(auto_ticket_node) — graph literally pauses mid-execution; resume continues from the pause point without re-running anything before it
- Fix 4: conditional skip flag — auto_ticket_node sets state['hitl_executed'] = True after Pass 2; downstream write agents check this flag before executing
- This is a real architectural tension: graph re-run simplicity vs precise resume control — LangGraph's interrupt API is the principled solution

---

Q12 Design

An interviewer asks: 'Why did you implement HITL inside LangGraph rather than outside it — e.g. the API pauses, stores the pending action in a separate DB table, and a webhook triggers the write when the user approves?' Defend your architectural choice.

Hint: Compare the in-graph vs out-of-graph HITL approaches.

Key points to cover:

- Out-of-graph HITL: FastAPI /query returns pending_action; a separate /approve endpoint receives approval; writes to a 'pending_actions' DB table; a worker polls and executes
- Out-of-graph advantages: simpler to understand; no LangGraph resume complexity; pending_actions table is easily queryable for ops monitoring
- Out-of-graph disadvantages: duplicates state — pending_action lives in BOTH the LangGraph checkpoint AND the pending_actions table; synchronisation risk
- Out-of-graph disadvantages: the second execution (worker) loses the full AgentState context — must re-fetch anomalies, agent_results from the DB table
- In-graph HITL advantages: state is in ONE place (PostgresSaver checkpoint) — single source of truth; no synchronisation needed
- In-graph HITL advantages: Pass 2 has FULL context from Pass 1 (all agent_results, sources, anomalies) — synthesizer can produce a richer post-approval response
- In-graph HITL advantages: thread_id continuity — the conversation history includes both the anomaly detection and the ticket creation in sequence
- Verdict: in-graph HITL is the right choice for this system because state continuity and context preservation are critical for a governance audit trail — the checkpoint IS the audit log

---